



Slide 1

CS 170

Java Programming 1 Actual Parameters

Duration: 00:01:17

Advance mode: Auto

Notes:

Hi Folks. Welcome to the CS 170, Java Programming 1 lecture on Actual Parameters. In the last lecture you learned to create simple components using a constructor. Like Henry Ford's Model T, which you could "order in any color you liked, as long as it was black", each object you've created from a single class has been identical; every object has had the same fields with the same starting values.

In this lesson you'll learn how to customize the objects you create by providing them with different starting values which they can use to initialize their data members. Then, instead of plain black, you'll be able to order your car with a nice candy-apple-red paint job. The values that you supply are called actual parameters.

As we work through this lecture, I'll ask you to create a new section in your "in-class" exercise lab document. For Exercise 1, open BlueJ and create a new, empty project named ic04. Then, snap me a screen-shot of the empty project. Our examples in this lecture will use Code Pad, but later online lectures this week will ask you to create some new classes in the ic04 project.

Object Variable Review

- For every **object variable** you describe:
 - What specific **kind** of thing the variable points to
 - What **name** you want to use for the variable

Color hairColor;

Construction Parameters

specify the **object** it refers to

you can create an object itself in several different ways

```

hairColor = new Color(128, 255, 128);
hairColor = new Color(.25F, .75F, 1.0F);
hairColor = Color.gray;
hairColor = someButton.getBackground();
  
```

Slide 2

Object Variable Review

Duration: 00:02:23

Advance mode: Auto

Notes:

Let's start with a little review from the last lesson, where you learned to create object variables and objects, and how to associate an object variable with a specific object. First, let's talk about the object variable itself. For every object variable you have to specify two things:

- first, the specific kind of thing that the variable points to. Here, our variable will point to a Color object. We won't be able to use this variable with a String, a Rectangle or a number. This is the data type of the variable.
- Second, you need to give the variable a name. Note that objects themselves are un-named, only variables are named.
- Once you've created a variable, you'll need to associate that variable with a specific object by storing the reference or address of the object inside the object variable using assignment. So, how do you get a reference to an object? There are several different ways.
- The most common way is to use the new operator and a constructor. Here are two different constructors for the Color class. Notice that both constructors have the same name (since all constructors must have the same name as their class), but that the information you pass to the constructor is of different types. When you have two versions of the same kind of constructor or method, we say that these are overloaded constructors or methods.

- You can also get a reference by referring to a pre-existing object. The Color class, for instance, automatically creates pre-defined color objects, called class constants. We can set `hairColor` to point to one of those pre-defined, such as `Color.gray`.
- Finally, we can ask another object to give us a reference to an object that it contains. Here, for instance, we ask a Button object to give use a reference to the Color object that the button uses to paint its background.

Look back now at the first of these examples, the one that uses the constructor. The three values, 128, 255, and 128 are construction parameters that the constructor will use to set the initial state of the Color object when it is created.

Let's take a look at what we mean by the term parameter.

What are Parameters?

- In the last lecture, you learned to construct a `Circle` object, and send a message to it, like this:

```
Circle c = new Circle(); // variable and object
c.makeVisible();        // send a message
```

- Note that we don't supply any additional information
- **Most** methods or constructors need extra information
 - A square root method needs the number to operate on
- Sent to your method via "**parameters**" or "**arguments**"

Slide 3

What are Parameters?

Duration: 00:02:03

Advance mode: Auto

Notes:

At the end of the last lecture, you used Code Pad to construct some Circle objects, and then make them visible. The code that you wrote looked something like this.

`Circle c = new Circle();` // This creates a new variable as well as a new object, and then assigns the object reference to the variable `c`.

`c.makeVisible();` // sends a message to your object, via the variable `c`

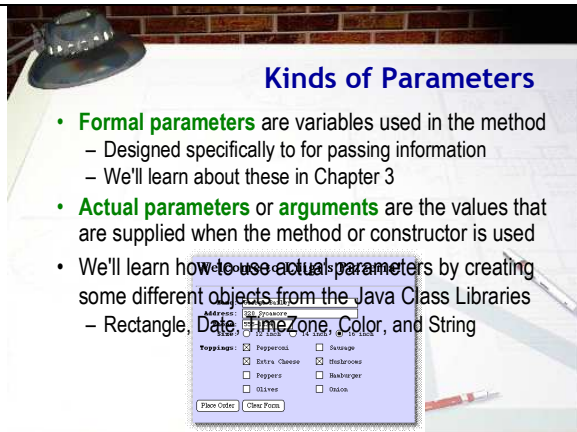
Now the important thing to notice here, is that you couldn't customize your Circle objects at all. Every single Circle you create is red, invisible, with a diameter of 30 pixels and located at 20x and 60 y.

Most constructors and most methods don't work this way, however; most require you to supply some extra information before they can carry out their work. Don't you think that the Circle class would be much more useful if you could specify the initial diameter and position, and maybe even the color? Can you imagine a square root calculation method didn't require you to supply the number to operate on? No! A square root method that just picked a number at random would be useless.

In these kinds of situations, we supply that extra information using parameters or arguments. There are actually two kinds of parameters, and your book uses the term "parameter" for both of them. Other books use "parameters" for one type, and "arguments" for the other. Still other books use the term "argument" for both. For our purposes, we'll consider the terms identical. Since your book uses the term parameter, I'll try to use that, but I may slip up and use the term argument, which is what I grew up with.

Unfortunately, when we use the term parameter, we use it for two related, but distinct concepts, and it's important to distinguish between the two when you are writing your programs.

Let's look at those two concepts now.



Kinds of Parameters

- **Formal parameters** are variables used in the method
 - Designed specifically to for passing information
 - We'll learn about these in Chapter 3
- **Actual parameters** or **arguments** are the values that are supplied when the method or constructor is used
- We'll learn how to use actual parameters by creating some different objects from the Java Class Libraries
 - Rectangle, Date, TimeZone, Color, and String

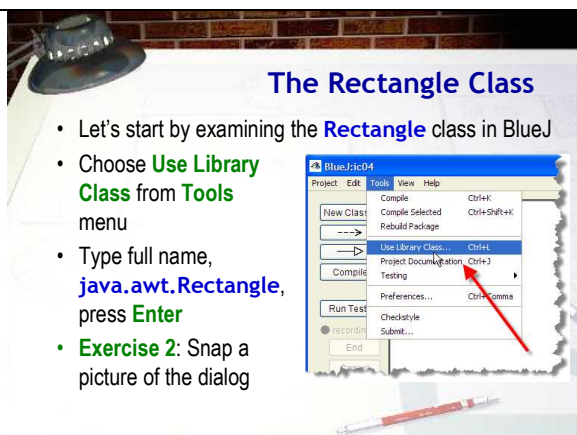
Slide 4
Kinds of Parameters
Duration: 00:01:27
Advance mode: Auto

Notes:

There are two kinds of parameters that we'll use in our programs. Formal parameters (or formal arguments) are the variables that you'll create to store information that will be used to hold the information supplied by the user. Let me give you an analogy. If you were designing an online pizza ordering form, the formal parameters would be the text fields, radio buttons and checkboxes that the user would use to fill in when ordering a pizza. Formal parameters are specifically designed for passing information into a method or a constructor. You'll learn how to create formal parameters in Chapter 3, when you start designing your own classes.

Actual parameters or arguments are the values supplied when you create an object or when you call a method. In our Luigi's Pizzeria analogy, the actual parameters are the values typed into the textboxes (George Bailey) or selected in the radio buttons and checkboxes (16 inch pepperoni with mushrooms and extra cheese).

In this lecture we'll learn how to use actual parameters by creating some objects from pre-existing classes: the classes in the Java Class Libraries that come with Java. In this lecture we'll work with the Rectangle class; in the next few lectures you'll use the Date, TimeZone, Color and String classes.



The Rectangle Class

- Let's start by examining the **Rectangle** class in BlueJ
- Choose **Use Library Class** from **Tools** menu
- Type full name, **java.awt.Rectangle**, press **Enter**
- **Exercise 2:** Snap a picture of the dialog

Slide 5
The Rectangle Class
Duration: 00:00:58
Advance mode: Auto

Notes:

So, how do we use a class when our project (ic04) doesn't have any classes displayed? (Unlike the Shapes project that we used in the last lecture.) You'll learn how to do that by exploring the Rectangle class.

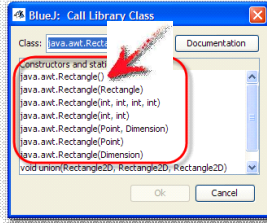
* First, on the Tools menu, choose Use Library Class

* When the Call Library Class dialog appears, type the full name of the class we want to work with: java.awt.Rectangle into the drop-down list labeled Class at the top of the dialog. Make sure you capitalize the R in Rectangle, and spell the rest of the name exactly as it appears on the slide. Once you've typed that in, press the Enter key to submit it. (There is no separate "Submit" or "Go" button.)

For Exercise 2, snap me a picture of the dialog, after you hit Enter.

Different Constructors

- Notice that there are seven **different** versions of the **Rectangle** constructor (*overloaded constructors*)
 - Each requires a different type or number of values
- 1 with no parameters
- 4 with object parameters
- 2 with **int** parameters
- What do those represent
- Click the **Documentation** Button



Slide 6

Different Constructors

Duration: 00:01:50

Advance mode: Auto

Notes:

When you select a specific class from the Java Class Libraries, the list box right below shows all of the constructors as well as the static or class methods. We'll ignore the class methods for right now, and concentrate on the constructors.

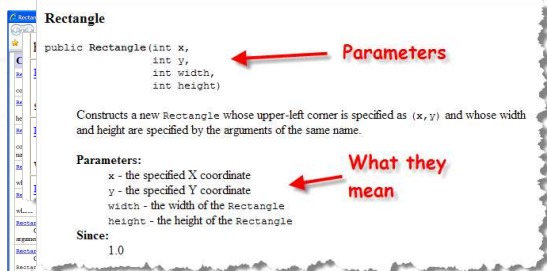
Notice that there are seven different versions of the Rectangle constructor. As you recall, the term we use to describe this is overloaded constructor. Each constructor is different because it has a different number or different kinds of parameters. In other words, you'll supply different values when you use each of these constructors.

- One constructor takes no parameters. These work like the Circle constructor you used in the last lesson. The class itself will determine what values each Rectangle should have when it is created, and every Rectangle created using this constructor will have the same initial values.
- Four constructors take different kinds of object parameters: one that takes a Rectangle, one that takes a Point, one that takes a Dimension, and one that takes a Point along with a Dimension.
- Two of the constructors take int parameters. One requires you to supply two int parameters, which you know how to do, and one requires you to supply four int parameters, which you also know how to do.

Here's the problem, though. The Call Library Class dialog tells you what kind of information you need to supply to use each constructor. What it doesn't tell you is what those ints or Points or Rectangles represent. To get that information, you'll need to look at the Java documentation, which you can reach by just clicking the Documentation button in the upper-right corner of the dialog.

Reading the JavaDocs

- Opens your browser to a screen showing more information about each constructor's parameters



Slide 7

Reading the JavaDocs

Duration: 00:02:05

Advance mode: Auto

Notes:

When you click the Documentation button, BlueJ opens your Web browser and links you to the API documentation on Sun's site. (API is some computer-speak jargon that stands for Application Programming Interface. In this case, it simply means the documentation for the class libraries.) I'll normally call this documentation the JavaDocs.

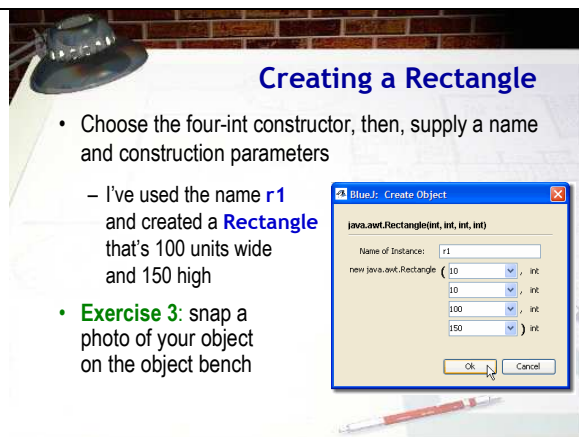
Using BlueJ you don't have to navigate to the specific area you want; BlueJ automatically looks up the correct page and section. Since we were trying to construct a Rectangle object, BlueJ opened the JavaDocs to the Constructor Summary for the Rectangle class.

Notice that the same seven constructors are listed here as were listed in the Call Class Library dialog, except this time there is some useful information. For instance, if you look at the constructor that takes two int parameters, you'll see that the JavaDocs name those parameters width and height,

which lets you know how the values you supply are used. For each of the constructors, there is also a single sentence or short paragraph that describes what the method does. This is similar to the sentence you need to add as a comment before each method you define.

Finally, if that's not enough, you can click the method hyperlink (the name of the method in the Summary section) and you'll be taken to the method detail section. Find the constructor that takes four int parameters, and click it now. Here, you'll find more information, specifically information on what each of the parameters mean. As you can see, with the four parameter constructor the first parameter is the X coordinate of the rectangle, the second the Y coordinate, the third the width and the last is the height.

Using that information, we can construct a Rectangle object.



Creating a Rectangle

- Choose the four-int constructor, then, supply a name and construction parameters
 - I've used the name **r1** and created a **Rectangle** that's 100 units wide and 150 high
- Exercise 3:** snap a photo of your object on the object bench

The screenshot shows the 'BlueJ: Create Object' dialog box for the class `java.awt.Rectangle`. The 'Name of Instance' field is set to `r1`. The parameters are: `x` (10), `y` (10), `width` (100), and `height` (150). The 'Ok' button is highlighted.

Slide 8

Creating a Rectangle

Duration: 00:00:48

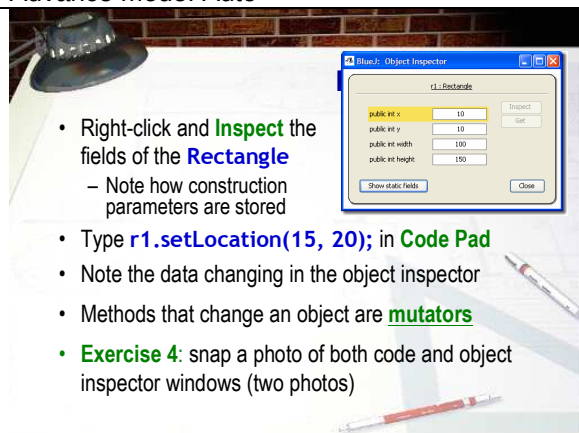
Advance mode: Auto

Notes:

Back on the Call Library Class dialog, select the constructor that requires 4 int parameters and click OK. When you do, BlueJ will open the Create Object dialog that looks like this. Note that there is a field for you to supply a name for the object variable, and four fields for each of the parameters. Supply the values that you'd like to use, and click the OK button.

In my example here, for instance, I've used the name `r1`, given it `x` and `y` values of 10, and made it 100 units wide and 150 units high.

Once you've created your object, for Exercise 3, snap me a picture of your new object on the object bench.



Rectangle Data

- Right-click and **Inspect** the fields of the **Rectangle**
 - Note how construction parameters are stored
- Type `r1.setLocation(15, 20);` in **Code Pad**
- Note the data changing in the object inspector
- Methods that change an object are **mutators**
- Exercise 4:** snap a photo of both code and object inspector windows (two photos)

The screenshot shows the 'BlueJ: Object Inspector' window for the object `r1: Rectangle`. It displays the following fields: `public int x` (10), `public int y` (10), `public int width` (100), and `public int height` (150). There are 'Inspect' and 'Get' buttons, and a 'Show static fields' button.

Slide 9

Rectangle Data

Duration: 00:01:15

Advance mode: Auto

Notes:

Well, let's see if the constructor actually worked. Right click your object in the object bench, and choose Inspect from the menu. Notice the fields: `x`, `y`, `width` and `height`, and how the values you supplied have been stored by the constructor into the appropriate fields.

Now, in Code Pad type:

```
r1.setLocation(15, 20);
```

(Use the name that you selected for your variable. If you named it something other than `r1`, then use that name instead.)

After you press Enter, notice that the values in each of the Object Inspector windows changes. The `x` field should now contain 15 and the `y` field 20. Methods like `setLocation()` are called mutator methods because they change the internal state of an object.

In the next lecture, we'll look at different kinds of methods.

	For right now, let's finish up this lecture by snapping me a picture of the code you typed in Code Pad and another picture of the Object Inspector window.
--	---